

[< Go to the original](#)

[illegible]

AI Agents in 5 Levels of Difficulty (With Full Code Implementation)

**Paolo Perrone**

Follow



Data Science Collective a11y-light ~6 min read · May 19, 2025 (Updated: May 19, 2025) · Free: Yes

It looked fine. It fetched data, called tools, and even explained its steps. But under the hood, it was bluffing. No real state, no memory, no reasoning. Just looping prompts pretending to be smart.

I only noticed when an edge case completely threw it off. That's when it hit me: I hadn't built an agent. I'd built a fancy prompt chain.

Fixing it meant redesigning the whole thing — not just chaining calls, but managing state, decisions, and long-term flow. Once that clicked, everything got simpler. The code, the logic, the results.

That's what this guide is about: breaking agent design into five practical levels of difficulty — each with working code.

The levels are:

- **Level 1: Agent with Tools and Instructions**
- **Level 2: Agent with Knowledge and Memory**
- **Level 3: Agent with Long-Term Memory and Reasoning**
- **Level 4: Multi-Agent Teams**
- **Level 5: Agentic Systems**

Alright, let's dive in.

Level 1: Agent with Tools and Instructions

This is the basic setup — an LLM that follows instructions and calls tools in a loop. When people say, "*agents are just LLMs plus tool use*," they're talking about this level (and revealing how far they've explored).

Instructions tell the agent what to do. Tools let it take action — fetching data, calling APIs, or triggering workflows. It's simple, but already powerful enough to automate some tasks.

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.tools.duckduckgo import DuckDuckGoTools

agno_assist = Agent(
    name="Agno AGI",
    model=OpenAIChat(id="gpt-4.1"),
    description=dedent("""\
    You are "Agno AGI, an autonomous AI Agent that can build agents using the Agno
    framework. Your goal is to help developers understand and use Agno by providing
    explanations, working code examples, and optional visual and audio explanations
    of key concepts."""),
    instructions="Search the web for information about Agno.",
    tools=[DuckDuckGoTools()],
    add_datetime_to_instructions=True,
    markdown=True,
)

agno_assist.print_response("What is Agno?", stream=True)
```

Level 2: Agent with Knowledge and Memory

Most tasks require information the model doesn't have. You can't stuff everything into context, so the agent needs a way to fetch knowledge at runtime — this is where agentic RAG or dynamic few-shot prompting comes in.

Storage gives the agent memory. LLMs are stateless by default; storing past actions, messages, and observations makes the agent stateful — able to reference what's happened so far and make better decisions.

Copy

```
... imports
# You can also use https://docs.agno.com/llms-full.txt for the full documentat
knowledge_base = UrlKnowledge(
    urls=["https://docs.agno.com/introduction.md"],
    vector_db=LanceDb(
        uri="tmp/lancedb",
        table_name="agno_docs",
        search_type=SearchType.hybrid,
        embedder=OpenAIEmbedder(id="text-embedding-3-small"),
        reranker=CohereReranker(model="rerank-multilingual-v3.0"),
    ),
)
storage = SqliteStorage(table_name="agent_sessions", db_file="tmp/agent.db")

agno_assist = Agent(
    name="Agno AGI",
    model=OpenAIChat(id="gpt-4.1"),
    description=...,
    instructions=...,
    tools=[PythonTools(), DuckDuckGoTools()],
```

```
knowledge=knowledge_base,  
# Store Agent sessions in a sqlite database  
storage=storage,  
# Add the chat history to the messages  
add_history_to_messages=True,  
# Number of history runs  
num_history_runs=3,  
markdown=True,  
)  
  
if __name__ == "__main__":  
    # Load the knowledge base, comment after first run  
    # agno_assist.knowledge.load(recreate=True)  
    agno_assist.print_response("What is Agno?", stream=True)
```

Level 3: Agent with Long-Term Memory and Reasoning

Memory lets agents recall details across sessions — like user preferences, past actions, or failed attempts — and adapt over time. This unlocks personalization and continuity. We're just scratching the surface here, but what excites me most is self-learning: agents that refine their behavior based on past experiences.

Reasoning takes things a step further.

increasing the success rate of each step. Every serious agent builder needs to know when and how to apply it.

Copy

```
... imports

knowledge_base = ...

memory = Memory(
    # Use any model for creating memories
    model=openAIChat(id="gpt-4.1"),
    db=SqliteMemoryDb(table_name="user_memories", db_file="tmp/agent.db"),
    delete_memories=True,
    clear_memories=True,
)

storage =

agno_assist = Agent(
    name="Agno AGI",
    model=Claude(id="claude-3-7-sonnet-latest"),
    # User for the memories
    user_id="ava",
    description=...,
    instructions=...,
    # Give the Agent the ability to reason
```

```
...  
# Store memories in a sqlite database  
memory=memory,  
# Let the Agent manage its memories  
enable_agentic_memory=True,  
)  
  
if __name__ == "__main__":  
    # You can comment this out after the first run and the agent will remember  
    agno_assist.print_response("Always start your messages with 'hi ava'", stream=True)  
    agno_assist.print_response("What is Agno?", stream=True)
```

Level 4: Multi-Agent Teams

Agents are most effective when they're focused — specialized in one domain with a tight toolset (ideally under 10). To tackle more complex or broad tasks, we combine them into teams. Each agent handles a piece of the problem, and together they cover more ground.

But there's a catch: without strong reasoning, the team leader falls apart on anything nuanced. Based on everything I've seen so far, autonomous multi-agent systems still don't work reliably. They succeed less than half the time — which isn't good enough.

with built-in memory and context management. You still need to design carefully, but these building blocks make serious multi-agent work more feasible.

Copy

```
... imports

web_agent = Agent(
    name="Web Search Agent",
    role="Handle web search requests",
    model= OpenAIChat(id="gpt-4o-mini"),
    tools=[DuckDuckGoTools()],
    instructions="Always include sources",
)

finance_agent= Agent(
    name="Finance Agent",
    role="Handle financial data requests",
    model=OpenAIChat(id="gpt-4o-mini"),
    tools=[YFinanceTools()],
    instructions=[
        "You are a financial data specialist. Provide concise and accurate data.",
        "Use tables to display stock prices, fundamentals (P/E, Market Cap)",
    ],
)
```

```

mode= Coordinate ,
model=Claude(id="claude-3-7-sonnet-latest"),
members=[web_agent, finance_agent],
tools=[ReasoningTools(add_instructions=True)],
instructions=[
    "Use tables to display data",
    "Only output the final answer, no other text.",
],
show_members_responses=True,
enable_agentic_context=True,
add_datetime_to_instructions=True,
success_criteria="The team has successfully completed the task.",
)

if __name__ == "__main__":
    team_leader.print_response(
        """\
        Analyze the impact of recent US tariffs on market performance across
these key sectors:
- Steel & Aluminum: (X, NUE, AA)
- Technology Hardware: (AAPL, DELL, HPQ)

For each sector:
1. Compare stock performance before and after tariff implementation
2. Identify supply chain disruptions and cost impact percentages
3. Analyze companies' strategic responses (reshoring, price adjustments, supply
diversification)""",
        stream=True,
    )

```

Level 5: Agentic Systems

This is where agents go from being tools to infrastructure. Agentic Systems are full APIs — systems that take in a user request, kick off an async workflow, and stream results back as they become available.

Sounds clean in theory. In practice, it's hard. Really hard.

You need to persist state when the request comes in, spin up a background job, track progress, and stream output as it's generated. Websockets can help, but they're tricky to scale and maintain. Most teams underestimate the backend complexity here.

This is what it takes to turn agents into real products. At this level, you're not building a feature — you're building a system.

From Demo Fails to Real Wins: Key Lessons in Agent Design

asynchronous agentic systems, adds power *only* when the underlying architecture is sound.

Most failures don't come from missing the latest framework. They come from ignoring the basics: clear boundaries, solid reasoning, effective memory, and knowing when to let humans take the wheel.

If you start simple, build up with purpose, don't overcomplicate upfront and add complexity only when it solves a real problem, you won't just build something cool — you'll build something that works.

Want to hear from me more often?

👉 [Connect with me on LinkedIn!](#)

I share *daily* actionable insights, tips, and updates to help you avoid costly mistakes and stay ahead in the AI world. Follow me here:

Are you a tech professional looking to grow your audience through writing?

My **Tech Audience Accelerator** is packed with actionable copywriting and audience building strategies that have helped hundreds of professionals stand out and accelerate their growth. Subscribe now to stay in the loop.



Freedium

[ko-fi](#) [librepat](#) [patreon](#)
